

EdgeLLMHub Developer Onboarding Guide

Section 1: Welcome & How to Use This Guide

What EdgeLLMHub Is

EdgeLLMHub is a clinical AI documentation system that enables doctors to dictate patient consultations, transcribe speech to text, and enrich transcripts into structured clinical notes. The system prioritizes patient privacy (PHI protection) by processing data on-device whenever possible, falling back to cloud LLM (Ollama) only when explicitly enabled, with a stub adapter for offline resilience.

How to Navigate This Document

This guide is organized by architecture (Sections 2-4) and implementation (Sections 5-9). Start with Section 2 for high-level context, then use Section 5 as a feature map. Section 7 is a quick-reference for file locations. Section 8 tells you how to run the project locally.

Prerequisites

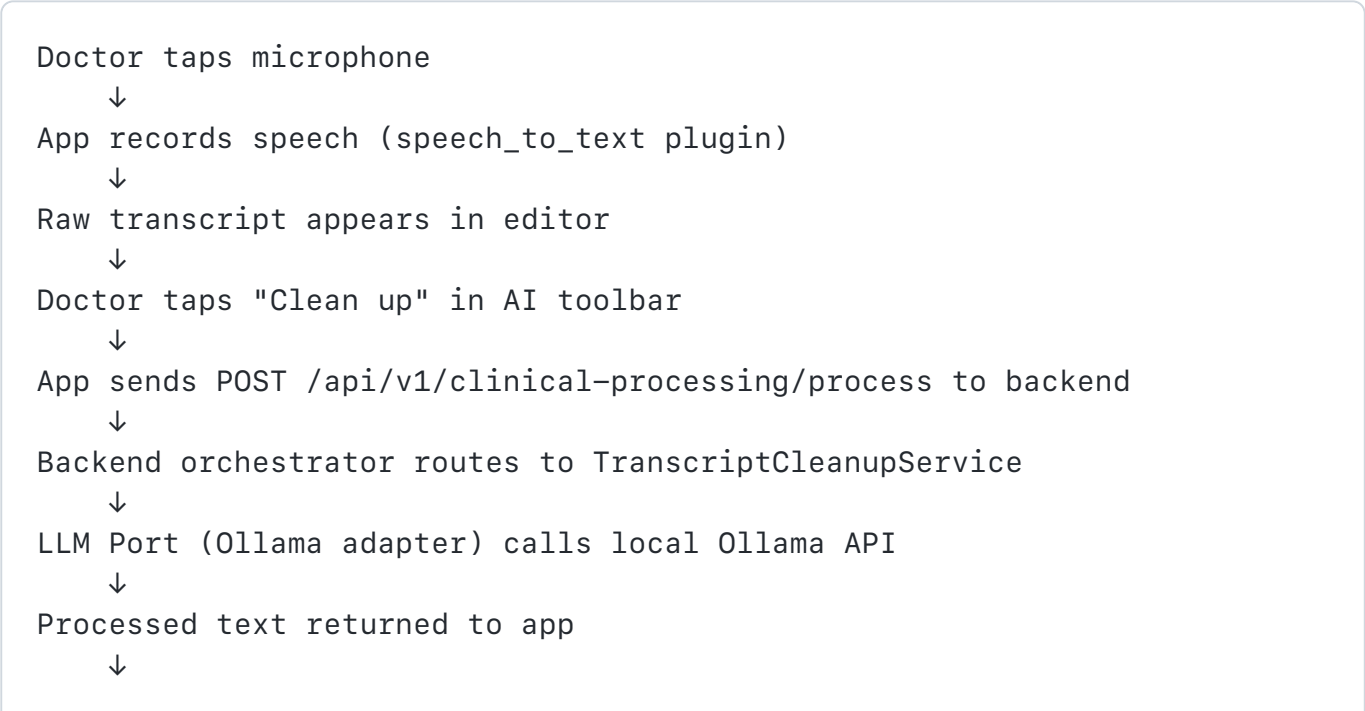
- **Dart 3.x** and **Flutter 3.x** (for `doctor_app`)
- **Dart Frog CLI** (for `clinical-intelligence-dart` backend)
- **Ollama** (for local LLM inference) — optional, stub mode works without it
- Basic understanding of **REST APIs**, **state management in Flutter**, and **dependency injection**

Section 2: Project Overview

The Two Apps

App	Type	Purpose	Architecture	Tech Stack
doctor_app	Flutter mobile app	Doctor dictates speech → app transcribes → AI cleans/enhances → structured clinical note is saved	Clean Architecture	Flutter, BLoC/Cubit, Drift (SQLite), Dio, speech_to_text plugin, flutter_gemma (Android)
clinical-intelligence-dart	Dart Frog backend	Receives text processing requests from mobile app, routes via hexagonal architecture, delegates to Ollama or stub	Hexagonal / Ports-and-Adapters	Dart Frog, http , provider() DI

User Journey



Doctor accepts suggestion → merged into note

↓

Note auto-saved to local SQLite (Drift)

LLM Strategy (PHI-Conscious)

Priority Order:

1. **On-device** (Android: Gemma via `flutter_gemma` / Google AI Edge; iOS: MLC) — PHI never leaves device
2. **Cloud** (Ollama via Dart Frog backend) — enabled only if `EnvironmentConfig.cloudLlmEnabled == true`
3. **Stub** (offline fallback) — returns static responses for basic functionality when offline

Fallback Logic: `HybridLlmAdapter._withFallback()` in `lib/core/llm/hybrid_llm_adapter.dart` approximately line 49 tries native first, then cloud (if permitted), then stub.

Technology Stack Summary

Component	doctor_app	clinical-intelligence-dart
Framework	Flutter	Dart Frog
State Management	BLoC/Cubit	—
DI	GetIt	<code>provider()</code> (Dart Frog context)
Persistence	Drift (SQLite)	—
HTTP Client	Dio	<code>http</code> package
LLM Integration	<code>flutter_gemma</code> (Android), <code>LlmPort</code> abstractions	<code>OllamaLlmAdapter</code> , <code>StubLlmAdapter</code>
Speech-to-Text	<code>speech_to_text</code> plugin	—

Section 3: Repository Structure

projects/apps/doctor_app/ (Flutter App)

```

projects/apps/doctor_app/
├── lib/
│   ├── main.dart                                # App entry point;
│   └── core/                                    # Shared utilities
│       ├── config/
│       │   └── environment.dart                # EnvironmentConf
│       ├── llm/
│       │   ├── llm_port_factory.dart          # LLM adapters and
│       │   │                                   # LlmPortFactory.cre
│       │   ├── hybrid_llm_adapter.dart        # HybridLlmAdapter.
│       │   ├── cloud_llm_adapter.dart         # CloudLlmAdapter:
│       │   ├── android_native_llm_adapter.dart # AndroidNativeLlmA
│       │   └── stub_llm_adapter.dart          # StubLlmAdapter: f
│       ├── network/
│       │   ├── retry_interceptor.dart         # RetryInterceptor:
│       │   └── circuit_breaker.dart           # CircuitBreaker: r
│       ├── ports/
│       │   └── llm_port.dart                  # LlmPort interface
│       └── services/
│           ├── local_speech_service.dart      # LocalSpeechService
│           └── speech_service_factory.dart    # SpeechServiceFacto
│
│   └── features/
│       └── note_assist/
│           ├── domain/
│           │   ├── entities/
│           │   ├── repositories/
│           │   └── services/
│           │       └── note_assist_service.dart # NoteAssistService
│           ├── data/
│           │   ├── local/
│           │   │   ├── note_local_repository.dart # NoteLocalReposit
│           │   │   └── local_database.dart        # Drift database s
│           │   ├── remote/
│           │   │   └── note_remote_datasource.dart # NoteRemoteDat
│           └── services/
│               └── on_device_llm_service.dart    # OnDeviceLlmServi

```

```

└─ presentation/                                # Presentation layer
    └─ cubit/
        └─ note_editor_cubit.dart                # NoteEditorCubit
        └─ ai_assist_cubit.dart                  # AiAssistCubit:
    └─ pages/
        └─ note_editor_page.dart                 # NoteEditorPage:
    └─ widgets/
        └─ ai_toolbar.dart                       # AiToolbar: "Clea
        └─ suggestion_panel.dart                 # SuggestionPanel
└─ ... (other app config)
└─ ... (Flutter assets, pubspec, etc.)

```

Architectural Layer Mapping:

- **Presentation Layer:** features/*/presentation/ (Cubits, Pages, Widgets)
- **Domain Layer:** features/*/domain/ (Entities, Repository interfaces, Use Cases as Services)
- **Data Layer:** features/*/data/ (Repository implementations, Data Sources, Plugins)
- **Core:** core/ (Shared abstractions: DI, Ports, Adapters, Network, Config)

projects/apps/clinical-intelligence-dart/ (Dart Frog Backend)

```

projects/apps/clinical-intelligence-dart/
└─ routes/                                # HTTP handlers
    └─ _middleware.dart                    # Middleware: provid
    └─ api/
        └─ v1/
            └─ clinical-processing/
                └─ process.dart              # onRequest(): parse
└─ lib/
    └─ application/                        # Application service
        └─ services/
            └─ clinical_processing_orchestrator.dart # ClinicalProcessing
            └─ terminology_assistance_service.dart # TerminologyAssistan
            └─ transcript_cleanup_service.dart      # TranscriptCleanup
    └─ infrastructure/                    # Adapters (driven s

```

```

├── llm/
│   ├── ollama_llm_adapter.dart
│   └── stub_llm_adapter.dart
├── domain/
│   └── dtos/
│       ├── clinical_processing_request.dart
│       └── clinical_processing_response.dart
├── interfaces/
│   └── llm_port.dart
└── ... (Dart Frog config, pubspec)

```

OllamaLlmAdapter:
StubLlmAdapter: no
Domain models

ClinicalProcessing
ClinicalProcessing
Ports (driving side)
LlmPort: processT

Architectural Layer Mapping:

- **Handlers:** routes/ (HTTP layer)
- **Application:** lib/application/ (Orchestrators + Services)
- **Domain:** lib/domain/ (DTOs, pure logic)
- **Infrastructure:** lib/infrastructure/ (Adapters for external systems)
- **Interfaces:** lib/interfaces/ (Ports/abstractions)

Section 4: Architecture Deep Dive

4a. doctor_app — Clean Architecture

The Three Rings (Dependency Rule: Inner → Outer only)

PRESENTATION (UI)

└ Pages, Widgets, Cubits

└ Depends on: Domain (interfaces) + Data (concrete)

↓ (calls)

DOMAIN

└ Entities (DoctorNote), Repository Interfaces, Use Cases

```
└ Pure Dart; NO Flutter/Drift/Dio dependencies
```

↓ (implements)

```
DATA
```

```
└ Repository Implementations, Data Sources, Plugins
└ Depends on: Domain (interfaces) + Core (abstractions)
```

↓ (uses)

```
CORE
```

```
└ Shared abstractions: LlmPort, LlmPortFactory, Environment
└ No business logic
```

BLoC/Cubit: How State Flows

- **Cubit** = simplified BLoC with single state stream (no events). Example:
NoteEditorCubit owns NoteEditorState which wraps a DoctorNote .
- **Flow:**
 - i. UI triggers action (e.g., `_toggleListening()` in `note_editor_page.dart` line 52)
 - ii. Cubit method called (`NoteEditorCubit.updateText()` in `note_editor_cubit.dart` line 88)
 - iii. Cubit updates state via `emit(newState)` (line 96 in `note_editor_cubit.dart`)
 - iv. Flutter rebuilds widgets listening to `BlocBuilder<NoteEditorCubit, NoteEditorState>`

GetIt: Dependency Injection Container

- **Registration:** `setupDependencies()` in `main.dart` lines 64-136 registers:
 - Dio with BaseOptions and RetryInterceptor (lines 82-90)
 - CircuitBreaker (line 79) — **registered but unused**
 - `LlmPortFactory.create()` → `HybridLlmAdapter` (line 111)
 - Repository implementations and services
- **Resolution:** `GetIt.I<SpeechService>()` in `note_editor_page.dart` line 59

- **Rule:** Register abstract interfaces → concrete implementations

Drift: Reactive SQLite Persistence

- **Schema:** `LocalDatabase` in `local_database.dart` defines `doctorNotes` table (line 11) with `primaryKey()` (line 24)
- **Access:** `NoteLocalRepository.saveNote()` in `note_local_repository.dart` line 11 calls
`db.into(db.doctorNotes).insertOnConflictUpdate(DoctorNotesCompanion(...))` (line 12)
- **Connection:** `_openConnection()` in `local_database.dart` line 91 calls
`NativeDatabase.createInBackground(file)`
- **Note:** No dedicated DAO class; repository writes directly through `LocalDatabase`

4b. clinical-intelligence-dart — Hexagonal Architecture

Ports-and-Adapters (Driving vs Driven)

- **Driving Side (App → Core):** `lib/interfaces/llm_port.dart` defines `LlmPort` interface with `processText(inputText, processingMode)`
- **Driven Side (Core → External):** Adapters implement `LlmPort` :
 - `OllamaLlmAdapter` (`infrastructure/llm/ollama_llm_adapter.dart`) — HTTP to local Ollama
 - `StubLlmAdapter` (`infrastructure/llm/stub_llm_adapter.dart`) — Static responses

Dart Frog `provider()` DI Chain

- **Middleware:** `middleware()` in `routes/_middleware.dart` line 24:
 - Constructs `OllamaLlmAdapter()` (line 26) → assigned to `LlmPort llmPort`
 - Constructs `TerminologyAssistanceService(llmPort, ...)` (line 39)
 - Constructs `TranscriptCleanupService(llmPort, ...)` (line 44)
 - Constructs `ClinicalProcessingOrchestrator(validationService, terminologyAssistanceService, transcriptCleanupService)` (line 57)
 - Registers via `provider<ClinicalProcessingOrchestrator>(() =>`

orchestrator) (line 76)

- **Handler:** onRequest() in process.dart line 18 calls
context.read<ClinicalProcessingOrchestrator>() (line 44)
- **Flow:** Request → Route Handler → provider() → Orchestrator → Services → LlmPort → Adapter

Why the Orchestrator Exists

- **Separation of Concerns:** Orchestrator
(clinical_processing_orchestrator.dart) routes ProcessingMode to the correct service:
 - ProcessingMode.vocabAssist → TerminologyAssistanceService (line 37)
 - ProcessingMode.cleanTranscript → TranscriptCleanupService (line 38)
- **Validation:** Orchestrator validates ClinicalProcessingRequest via
ValidationService (line 33) before processing
- **PHI Protection:** Ensures LLM prompts are wrapped with clinical context and
validation happens before external calls

4c. How the Two Apps Connect

HTTP Boundary

—— HTTP BOUNDARY ——

APP sends: POST /api/v1/clinical-processing/process

body: { inputText: String, processingMode: String }

(from CloudLlmAdapter._post() in doctor_app/lib/core/llm/cloud_

BACKEND receives at: routes/api/v1/clinical-processing/process.dart line 1

Shared DTO Contract

DTO	Fields	Direction	Status
	inputText: String ,	App →	

ClinicalProcessingRequest	processingMode: String	Backend	fromJson() implemented
ClinicalProcessingResponse	processedText: String , processingMode: String , warnings: List<String> , generatedAt: String , metadata: Object	Backend → App	⚠ toJson() exists; fromJson() MISSING

DTO Duplication Risk

- **Problem:** Both apps define their own copies of `ClinicalProcessingRequest` and `ClinicalProcessingResponse` (not in a shared package)
- **Risk:** Schema drift — if one app changes a field, the other may break at runtime
- **Mitigation:** Currently manual alignment; consider extracting to a shared Dart package

Section 5: Feature-by-Feature Wiring

Feature 1 — Voice Capture & Speech-to-Text

Plain English Summary

The doctor taps the microphone button in the note editor to start dictating. The app captures audio via the device microphone, uses the native `speech_to_text` plugin to transcribe speech in real-time, and streams recognized words back into the editor. Each word is appended to the note's raw text and persisted via the Cubit state.

Call Chain

STEP 1 Doctor taps microphone FAB to toggle recording

- WHO: FloatingActionButton.onPressed in lib/features/note_assist/pre
- CALLS: _toggleListening() on _NoteEditorPageState in lib/features/not
- WHY: The FAB is the UI entry point for voice capture; toggles listeni

STEP 2 Page flips listening state in Cubit

- WHO: _toggleListening() in lib/features/note_assist/presentation/pa
- CALLS: setListening() on NoteEditorCubit in lib/features/note_assist/
- WHY: Cubit state must reflect active dictation to update UI (micropho

STEP 3 Page starts injected SpeechService with transcript callback

- WHO: _toggleListening() in lib/features/note_assist/presentation/pa
- CALLS: startListening(Function(String)) on SpeechService in lib/featu
- WHY: Delegates speech capture to abstract service; callback receives

STEP 4 SpeechServiceFactory selects native implementation for capable dev

- WHO: SpeechServiceFactory.create() in lib/core/services/speech_serv
- CALLS: canUseSpeechToText() on DeviceCapabilityService in lib/core/se
- WHY: Physical devices use native STT; simulators fall back to cloud/m

STEP 5 LocalSpeechService initializes the speech_to_text plugin

- WHO: LocalSpeechService.initialize() in lib/core/services/local_spe
- CALLS: initialize() on stt.SpeechToText in lib/core/services/local_sp
- WHY: Plugin must report availability before starting native speech re

STEP 6 LocalSpeechService starts listening and receives recognized words

- WHO: LocalSpeechService.startListening() in lib/core/services/local
- CALLS: listen() on stt.SpeechToText in lib/core/services/local_speech
- WHY: Plugin delivers recognized text via onResult callback with resul

STEP 7 Recognized text is appended to editor and sent to Cubit

- WHO: _toggleListening() callback in lib/features/note_assist/presen
- CALLS: updateText(newText) on NoteEditorCubit in lib/features/note_as
- WHY: Dictated text must update both the TextEditingController and the

STEP 8 Cubit updates DoctorNote rawText and emits new state

- WHO: NoteEditorCubit.updateText() in lib/features/note_assist/prese
- CALLS: copyWith(rawText: newText) on DoctorNote in lib/features/note_
- WHY: NoteEditorLoaded state holds DoctorNote as source of truth; emit

Key Files to Open

- `lib/features/note_assist/presentation/pages/note_editor_page.dart`
- `lib/features/note_assist/presentation/cubit/note_editor_cubit.dart`
- `lib/core/services/local_speech_service.dart`
- `lib/core/services/speech_service_factory.dart`

Status Badge

 **FULLY WIRED**

Feature 2 — Clinical Processing Dispatch (App → Backend)

Plain English Summary

When the doctor taps "Clean up" in the AI toolbar, the app packages the raw transcript and sends it to the backend for AI-driven clinical text processing. The request is routed through a domain service, then through the LLM port abstraction, ultimately making an HTTP POST to the Dart Frog backend if cloud is enabled or on-device LLM is unavailable.

Call Chain

STEP 1 Doctor taps "Clean up" in AI toolbar

- WHO: `AiToolbar.build()` in `lib/features/note_assist/presentation/wid`
- CALLS: `cleanUpText(currentText)` on `AiAssistCubit` in `lib/features/note`
- WHY: UI action maps to the clinical clean-transcript workflow via Cub

STEP 2 `AiAssistCubit` starts generation via domain service

- WHO: `AiAssistCubit.cleanUpText()` in `lib/features/note_assist/presen`
- CALLS: `cleanUpText(rawText)` on `NoteAssistService` in `lib/features/note`
- WHY: Presentation layer delegates AI processing to domain service abs

STEP 3 `OnDeviceLlmService` translates action to LLM port call

- WHO: `OnDeviceLlmService.cleanUpText()` in `lib/features/note_assist/d`
- CALLS: `processText(rawText, ProcessingMode.cleanTranscript)` on `LlmPort`
- WHY: Maps domain intention (`cleanUpText`) to correct LLM processing mo

STEP 4 `HybridLlmAdapter` chooses native first, then cloud, then stub

- WHO: `HybridLlmAdapter.processText()` in `lib/core/llm/hybrid_llm_adap`
- CALLS: `_withFallback()` on `HybridLlmAdapter` in `lib/core/llm/hybrid_llm`

└─ WHY: Compliance and resilience enforced by native → cloud → stub fall

STEP 5 CloudLlmAdapter prepares backend POST

└─ WHO: CloudLlmAdapter.processText() in lib/core/llm/cloud_llm_adapte
 └─ CALLS: _post() on CloudLlmAdapter in lib/core/llm/cloud_llm_adapter.d
 └─ WHY: Cloud fallback serializes inputText and processingMode for backe

—— HTTP BOUNDARY ——

APP sends: POST /api/v1/clinical-processing/process

body: { inputText: string, processingMode: string }

BACKEND receives at: routes/api/v1/clinical-processing/process.dart line

STEP 6 Dio performs outgoing HTTP POST

└─ WHO: CloudLlmAdapter._post() in lib/core/llm/cloud_llm_adapter.dart
 └─ CALLS: post() on Dio in lib/core/llm/cloud_llm_adapter.dart line 87
 └─ WHY: Actual network call for /api/v1/clinical-processing/process with

STEP 7 Dio is constructed with shared API base URL and interceptors

└─ WHO: setupDependencies() in lib/main.dart line 64
 └─ CALLS: Dio() with BaseOptions(baseUrl: EnvironmentConfig.apiUrl) i
 └─ WHY: Centralized HTTP client configuration for backend API.

STEP 8 RetryInterceptor attached to shared Dio instance

└─ WHO: setupDependencies() in lib/main.dart line 90
 └─ CALLS: add() on dio.interceptors in lib/main.dart line 90
 └─ WHY: Transient idempotent HTTP failures automatically retried with ex

Key Files to Open

- lib/features/note_assist/presentation/widgets/ai_toolbar.dart
- lib/features/note_assist/presentation/cubit/ai_assist_cubit.dart
- lib/features/note_assist/data/services/on_device_llm_service.dart
- lib/core/llm/cloud_llm_adapter.dart
- lib/core/llm/hybrid_llm_adapter.dart

Status Badge



PARTIALLY WIRED

Gaps



GAP: Dispatch goes through CloudLlmAdapter directly; no

ClinicalProcessingRemoteDatasource exists — Fix: Consider introducing a repository pattern for clinical processing remote calls in
 lib/features/note_assist/data/remote/

Feature 3 — Backend: Request Reception & Orchestration

Plain English Summary

The Dart Frog backend receives the clinical processing POST request, parses the JSON body into a `ClinicalProcessingRequest` DTO, and delegates to the `ClinicalProcessingOrchestrator`. The orchestrator validates the request, routes it to the appropriate service based on `processingMode` (e.g., `CLEAN_TRANSCRIPT` → `TranscriptCleanupService`), which then calls the LLM port. The response is serialized as JSON and returned to the client.

Call Chain

```
STEP 1 Route handler receives and parses the request body
├─ WHO:   onRequest() in routes/api/v1/clinical-processing/process.dart
├─ CALLS: body() on Request in routes/api/v1/clinical-processing/process
└─ WHY:   Reads the HTTP request body as bytes for JSON parsing.

STEP 2 Route parses body into ClinicalProcessingRequest DTO
├─ WHO:   onRequest() in routes/api/v1/clinical-processing/process.dart
├─ CALLS: fromJson() on ClinicalProcessingRequest in routes/api/v1/clinical-processing/process
└─ WHY:   Deserializes JSON into typed request DTO for processing.

STEP 3 Route retrieves orchestrator from Dart Frog provider()
├─ WHO:   onRequest() in routes/api/v1/clinical-processing/process.dart
├─ CALLS: read<ClinicalProcessingOrchestrator>() on RequestContext in routes/api/v1/clinical-processing/process
└─ WHY:   Looks up the orchestrator instance injected by middleware.

STEP 4 Root middleware injects ClinicalProcessingOrchestrator
├─ WHO:   middleware() in routes/_middleware.dart line 24
├─ CALLS: provider<ClinicalProcessingOrchestrator>((context) => orchestrator) on Dart Frog DI
└─ WHY:   Provides the orchestrator to all route handlers via Dart Frog DI

STEP 5 Middleware constructs the active LLM dependency as Ollama
├─ WHO:   middleware() in routes/_middleware.dart line 24
├─ CALLS: OllamaLlmAdapter() on OllamaLlmAdapter in routes/_middleware.dart
```

└─ WHY: Provides concrete LLM adapter to services. StubLlmAdapter exists

STEP 6 Middleware constructs terminology and cleanup services

└─ WHO: middleware() in routes/_middleware.dart line 24

└─ CALLS: TerminologyAssistanceService(llmPort, normalizationService) in

└─ CALLS: TranscriptCleanupService(llmPort, normalizationService) in rou

└─ WHY: Services depend on LlmPort abstraction to keep LLM details out o

STEP 7 Middleware constructs orchestrator with services

└─ WHO: middleware() in routes/_middleware.dart line 24

└─ CALLS: ClinicalProcessingOrchestrator(validationService, terminologyAss

└─ WHY: Orchestrator composes validation and processing services.

STEP 8 Route invokes orchestration with parsed request

└─ WHO: onRequest() in routes/api/v1/clinical-processing/process.dart

└─ CALLS: process(request) on ClinicalProcessingOrchestrator in routes/a

└─ WHY: Delegates business logic to orchestrator layer.

STEP 9 Orchestrator validates request

└─ WHO: ClinicalProcessingOrchestrator.process() in lib/application/se

└─ CALLS: validateClinicalProcessingRequest() on ValidationService in li

└─ WHY: Ensures request is valid before processing (e.g., inputText not

STEP 10 VOCAB_ASSIST routes to terminology assistance

└─ WHO: ClinicalProcessingOrchestrator.process() in lib/application/se

└─ CALLS: _processVocabAssist() on ClinicalProcessingOrchestrator in lib

└─ WHY: Routes ProcessingMode.vocabAssist to terminology service.

STEP 11 Terminology assistance reaches LLM port

└─ WHO: TerminologyAssistanceService.process() in lib/application/serv

└─ CALLS: processText() on LlmPort in lib/application/services/terminolo

└─ WHY: Service uses LLM port abstraction for text processing.

STEP 12 CLEAN_TRANSCRIPT routes to transcript cleanup

└─ WHO: ClinicalProcessingOrchestrator.process() in lib/application/se

└─ CALLS: _processCleanTranscript() on ClinicalProcessingOrchestrator in

└─ WHY: Routes ProcessingMode.cleanTranscript to cleanup service.

STEP 13 Transcript cleanup reaches LLM port

└─ WHO: TranscriptCleanupService.process() in lib/application/services

└─ CALLS: processText() on LlmPort in lib/application/services/transcrip

└─ WHY: Service delegates to LLM port for actual text processing.

STEP 14 Orchestrator returns ClinicalProcessingResponse DTO

└─ WHO: ClinicalProcessingOrchestrator._processVocabAssist() in lib/ap

```
└ CALLS: ClinicalProcessingResponse(...) constructor in lib/application
└ WHY: Wraps processed text and metadata into response DTO for serializ
```

STEP 15 Route serializes response as JSON

```
└ WHO: onRequest() in routes/api/v1/clinical-processing/process.dart
└ CALLS: json() on Response in routes/api/v1/clinical-processing/proces
└ WHY: Converts ClinicalProcessingResponse DTO to JSON for HTTP respons
```

Key Files to Open

- routes/api/v1/clinical-processing/process.dart
- routes/_middleware.dart
- lib/application/services/clinical_processing_orchestrator.dart
- lib/application/services/terminology_assistance_service.dart
- lib/application/services/transcript_cleanup_service.dart

Status Badge

 FULLY WIRED

Feature 4 — LLM Adapter Selection & Text Processing

Plain English Summary

The backend selects a concrete LLM adapter (currently Ollama) via Dart Frog middleware and injects it into the orchestrator's services. The Ollama adapter prepares a clinical prompt based on the processing mode, then sends an HTTP POST to a local Ollama instance for text generation. A stub adapter exists but is not active in middleware.

Call Chain

```
STEP 1 Middleware constructs Ollama as the active LLM adapter
└ WHO: middleware() in routes/_middleware.dart line 24
└ CALLS: OllamaLlmAdapter() in routes/_middleware.dart line 26
└ WHY: Provides concrete LLM implementation to the application layer.
! BUG: Comment at line 27 says "To use Ollama ... uncomment" but line 26
```

STEP 2 Ollama adapter builds prompt for requested processing mode


```

└─ WHO:  OllamaLlmAdapter.processText() in lib/infrastructure/llm/ollam
└─ CALLS: _generate(prompt, model) on OllamaLlmAdapter in lib/infrastruc
└─ WHY:  Wraps input text with clinical context prompt specific to proces

```

STEP 3 Ollama adapter opens HTTP request to local Ollama API

```

└─ WHO:  OllamaLlmAdapter._generate() in lib/infrastructure/llm/ollama_
└─ CALLS: postUrl() on HttpClient in lib/infrastructure/llm/ollama_llm_a
└─ WHY:  Opens connection to Ollama's HTTP API (typically http://localhos

```

STEP 4 Ollama adapter sends model, prompt, stream flag, temperature

```

└─ WHO:  OllamaLlmAdapter._generate() in lib/infrastructure/llm/ollama_
└─ CALLS: write() on HttpClientRequest in lib/infrastructure/llm/ollama_
└─ WHY:  Sends JSON body with model name (e.g., "gemma:2b"), prompt, stre

```

STEP 5 Stub adapter exists but is inactive

```

└─ WHO:  StubLlmAdapter.processText() in lib/infrastructure/llm/stub_ll
└─ NOTE:  No provider in middleware selects StubLlmAdapter; Ollama is acti

```

Key Files to Open

- routes/_middleware.dart
- lib/infrastructure/llm/ollama_llm_adapter.dart
- lib/infrastructure/llm/stub_llm_adapter.dart

Status Badge

 FULLY WIRED

Gaps

 GAP: Misleading comment in routes/_middleware.dart line 27 contradicts active Ollama instantiation at line 26 — Fix: Update comment to reflect that Ollama is already active, or remove the comment entirely.

Feature 5 — Response Propagation (Backend → App UI)

Plain English Summary

The backend serializes the `ClinicalProcessingResponse` DTO as JSON and returns it to the app. The app's `CloudLlmAdapter` receives the response, extracts the `processedText` field directly from the JSON map (bypassing DTO deserialization due

to missing `fromJson`), and streams it to the `AiAssistCubit` . The Cubit emits states to signal suggestion progress, and the UI renders the processed text in a `SuggestionPanel` . When the doctor accepts, the text is merged into the note editor.

Call Chain

STEP 1 Orchestrator returns `ClinicalProcessingResponse` DTO

- └ WHO: `ClinicalProcessingOrchestrator._processVocabAssist()` in `lib/ap`
- └ CALLS: `ClinicalProcessingResponse(processedText: ..., processingMode: .`
- └ WHY: Encapsulates processed text and metadata for serialization.

STEP 2 Orchestrator returns `ClinicalProcessingResponse` for `CLEAN_TRANSCRI`

- └ WHO: `ClinicalProcessingOrchestrator._processCleanTranscript()` in `li`
- └ CALLS: `ClinicalProcessingResponse(...)` in `lib/application/services/cl`
- └ WHY: Same response structure for both processing modes.

—— HTTP BOUNDARY ——

BACKEND sends: 200 JSON { processedText: string, processingMode: string,
APP receives in: `lib/core/llm/cloud_llm_adapter.dart` line 87

STEP 3 App receives JSON map from Dio response

- └ WHO: `CloudLlmAdapter._post()` in `lib/core/llm/cloud_llm_adapter.dart`
- └ CALLS: `post()` on `Dio` in `lib/core/llm/cloud_llm_adapter.dart` line 87
- └ WHY: Dio deserializes JSON response into `Map<String, dynamic>`.

STEP 4 `CloudLlmAdapter` extracts `processedText` directly from map

- └ WHO: `CloudLlmAdapter.processText()` in `lib/core/llm/cloud_llm_adapte`
- └ CALLS: Direct map access `response['processedText']` on `Map<String, dynam`
- └ WHY: Only needs `processedText`; bypasses DTO instantiation due to miss
- ⚠ NOT WIRED: `ClinicalProcessingResponse.fromJson()` is missing in `lib/core`

STEP 5 `AiAssistCubit` emits suggestion states (in-progress → ready)

- └ WHO: `AiAssistCubit._startGenerationStream()` in `lib/features/note_as`
- └ CALLS: `emit(AiAssistSuggestionInProgress)` on `AiAssistCubit` in `lib/fea`
- └ CALLS: `emit(AiAssistSuggestionReady)` on `AiAssistCubit` in `lib/features`
- └ WHY: Notifies UI of processing state transitions.

STEP 6 `SuggestionPanel` renders processed text

- └ WHO: `SuggestionPanel.build()` in `lib/features/note_assist/presentati`
- └ CALLS: `Text(response.processedText)` widget in `lib/features/note_assis`
- └ WHY: Displays the AI-generated suggestion to the doctor for review.

STEP 7 Doctor accepts suggestion → merged into note editor

```
├─ WHO:  NoteEditorPage.build() onAccept callback in lib/features/note_
├─ CALLS: updateText(newText) on NoteEditorCubit in lib/features/note_as
└─ WHY:  Accepted suggestion becomes part of the note's rawText.
```

Key Files to Open

- `lib/application/services/clinical_processing_orchestrator.dart`
- `lib/core/llm/cloud_llm_adapter.dart`
- `lib/features/note_assist/presentation/cubit/ai_assist_cubit.dart`
- `lib/features/note_assist/presentation/widgets/suggestion_panel.dart`
- `lib/features/note_assist/presentation/pages/note_editor_page.dart`

Status Badge



PARTIALLY WIRED

Gaps



GAP: `ClinicalProcessingResponse.fromJson()` is missing in `lib/core/dto/clinical_processing_response.dart` — Fix: Add `fromJson(Map<String, dynamic> json)` method to deserialize backend responses, then use it in `CloudLlmAdapter.processText()` instead of direct map access.

Feature 6 — On-Device LLM Path (flutter_gemma / Gemma local inference)

Plain English Summary

On Android, the app uses Google AI Edge via the `flutter_gemma` plugin for on-device LLM inference. The `LlmPortFactory` creates a `HybridLlmAdapter` that prioritizes the `AndroidNativeLlmAdapter`. The hybrid adapter first attempts on-device inference, falls back to cloud if `EnvironmentConfig.cloudLlmEnabled` is true, and finally uses the stub adapter if both fail.

Call Chain

```
STEP 1  EnvironmentConfig defines cloud LLM toggle
├─ WHO:  EnvironmentConfig.cloudLlmEnabled in lib/core/config/environe
```

└─ WHY: Runtime flag controlling whether cloud fallback is permitted.

STEP 2 App setup calls LlmPortFactory to create LLM port

└─ WHO: setupDependencies() in lib/main.dart line 111

└─ CALLS: LlmPortFactory.create() on LlmPortFactory in lib/core/llm/llm_

└─ WHY: Centralizes LLM adapter construction with environment-based conf

STEP 3 Factory checks if running on simulator

└─ WHO: LlmPortFactory.create() in lib/core/llm/llm_port_factory.dart

└─ CALLS: isSimulator() on DeviceCapabilityService in lib/core/llm/llm_p

└─ WHY: Simulators cannot use native LLM; must fall back to cloud/stub.

STEP 4 Factory selects Android native adapter for physical Android device

└─ WHO: LlmPortFactory.create() in lib/core/llm/llm_port_factory.dart

└─ CALLS: AndroidNativeLlmAdapter() on AndroidNativeLlmAdapter in lib/co

└─ WHY: Selects Gemma-based on-device adapter when running on Android.

STEP 5 Factory returns HybridLlmAdapter wrapping native adapter

└─ WHO: LlmPortFactory.create() in lib/core/llm/llm_port_factory.dart

└─ CALLS: HybridLlmAdapter(nativeAdapter, cloudEnabled) in lib/core/llm/

└─ WHY: Hybrid adapter manages fallback chain: native → cloud (if enable

STEP 6 OnDeviceLlmService wired to hybrid LLM port

└─ WHO: setupDependencies() in lib/main.dart line 136

└─ CALLS: OnDeviceLlmService(LlmPort) in lib/main.dart line 136

└─ WHY: Domain service uses LLM port abstraction for text processing.

STEP 7 HybridLlmAdapter attempts native first

└─ WHO: HybridLlmAdapter._withFallback() in lib/core/llm/hybrid_llm_ad

└─ CALLS: processText() on AndroidNativeLlmAdapter in lib/core/llm/andro

└─ WHY: Prioritizes on-device LLM to preserve PHI and enable offline cap

STEP 8 AndroidNativeLlmAdapter calls _generate()

└─ WHO: AndroidNativeLlmAdapter.processText() in lib/core/llm/android_

└─ CALLS: _generate(prompt) on AndroidNativeLlmAdapter in lib/core/llm/a

└─ WHY: Prepares the prompt and delegates to the native plugin.

STEP 9 _FlutterGemmaProxy calls flutter_gemma plugin

└─ WHO: AndroidNativeLlmAdapter._generate() in lib/core/llm/android_na

└─ CALLS: getResponse(prompt) on FlutterGemmaProxy in lib/core/llm/andro

└─ WHY: Invokes the flutter_gemma plugin for on-device inference.

STEP 10 flutter_gemma plugin executes on-device inference

└─ WHO: FlutterGemmaProxy.getResponse() in lib/core/llm/android_native

└─ CALLS: FlutterGemmaPlugin.getResponse() on FlutterGemmaPlugin in lib/

└─ WHY: Uses Google AI Edge to execute the prompt directly on the Android device.

⚠️ NOT WIRED: Android model verification is TODO-only in `lib/features/model_manager/presentation/cubit/model_manager_cubit.dart`.

⚠️ NOT WIRED: Android model download is commented/TODO-only in `lib/features/model_manager/presentation/cubit/model_manager_cubit.dart`.

Key Files to Open

- `lib/core/llm/llm_port_factory.dart`
- `lib/core/llm/hybrid_llm_adapter.dart`
- `lib/core/llm/android_native_llm_adapter.dart`
- `lib/core/config/environment.dart`
- `lib/features/model_manager/presentation/cubit/model_manager_cubit.dart`

Status Badge

⚠️ **PARTIALLY WIRED**

Gaps

🔧 GAP: Android model verification is TODO-only — Fix: Implement model verification logic in `ModelManagerCubit` to check if the required Gemma model is available on-device before attempting inference.

🔧 GAP: Android model download is commented out — Fix: Uncomment and implement model download functionality in `ModelManagerCubit` to fetch the Gemma model from Google AI Edge when not present.

Feature 7 — Note Persistence (Local Storage via Drift)

Plain English Summary

Every change to the note's text (via dictation or manual editing) triggers an auto-save via the `NoteEditorCubit`. The cubit schedules a debounced save to the `NoteLocalRepository`, which maps the `DoctorNote` domain entity to a Drift `DoctorNotesCompanion` and upserts it into the `doctorNotes` SQLite table. Drift handles the database connection in the background.

Call Chain

STEP 1 `NoteEditorCubit` loads or creates note on initialization

```
| WHO: NoteEditorCubit.loadOrCreateNote() in lib/features/note_assist
| CALLS: DoctorNote() constructor in lib/features/note_assist/presentat
| CALLS: saveNote(note) on NoteLocalRepository in lib/features/note_ass
| WHY: Creates initial note state and persists it immediately.
```

STEP 2 NoteEditorCubit updates DoctorNote rawText on user input

```
| WHO: NoteEditorCubit.updateText() in lib/features/note_assist/prese
| CALLS: copyWith(rawText: newText) on DoctorNote in lib/features/note_
| WHY: Updates the domain entity's rawText field immutably.
```

STEP 3 NoteEditorCubit schedules auto-save

```
| WHO: NoteEditorCubit.updateText() in lib/features/note_assist/prese
| CALLS: _scheduleAutoSave() on NoteEditorCubit in lib/features/note_as
| WHY: Debounces save operations to avoid excessive disk writes.
```

STEP 4 NoteEditorCubit saves note to local repository

```
| WHO: NoteEditorCubit._saveNote() in lib/features/note_assist/presen
| CALLS: saveNote(note) on NoteLocalRepository in lib/features/note_ass
| WHY: Delegates persistence to the data layer.
```

STEP 5 NoteLocalRepository maps DoctorNote to Drift companion and upserts

```
| WHO: NoteLocalRepository.saveNote() in lib/features/note_assist/dat
| CALLS: insertOnConflictUpdate() on InsertStatement in lib/features/no
| WHY: Converts domain entity to Drift companion and performs upsert to
! NOT WIRED: No dedicated Drift DAO class; NoteLocalRepository writes dir
```

STEP 6 DoctorNotes table defined in Drift schema

```
| WHO: DoctorNotes table in lib/features/note_assist/data/local/local
| CALLS: primaryKey() in lib/features/note_assist/data/local/local_data
| WHY: Defines the SQLite table structure for notes with auto-increment
```

STEP 7 Generated Drift accessor handles queries

```
| WHO: _$LocalDatabase generated code in lib/features/note_assist/dat
| WHY: Drift code generation provides type-safe database access.
```

STEP 8 Database connection opened in background

```
| WHO: _openConnection() in lib/features/note_assist/data/local/local
| CALLS: NativeDatabase.createInBackground(file) in lib/features/note_a
| WHY: Opens SQLite connection asynchronously to avoid blocking the UI
```

Key Files to Open

- `lib/features/note_assist/presentation/cubit/note_editor_cubit.dart`

- `lib/features/note_assist/data/local/note_local_repository.dart`
- `lib/features/note_assist/data/local/local_database.dart`

Status Badge



PARTIALLY WIRED

Gaps



GAP: No dedicated Drift DAO class — **Fix:** Consider creating a `DoctorNoteDao` class in `lib/features/note_assist/data/local/` to encapsulate all Drift operations, then have `NoteLocalRepository` delegate to it for better separation of concerns.

Feature 8 — Android Emulator URL Resolution

Plain English Summary

When running the app on an Android emulator, the backend API URL needs to be `http://10.0.2.2:<port>` to reach the host machine's Dart Frog server. The `EnvironmentConfig` provides both `apiBaseUrl` (default) and `androidEmulatorApiUrl` (for emulators). However, the shared Dio instance in `main.dart` always uses `apiBaseUrl`, bypassing the emulator-specific URL.

Call Chain

```

STEP 1  EnvironmentConfig defines default API base URL
├─ WHO:   EnvironmentConfig.apiBaseUrl in lib/core/config/environment.da
├─ CALLS: fromEnvironment('API_URL', defaultValue: 'http://localhost:8080'
└─ WHY:   Determines the default backend API location from environment var

STEP 2  EnvironmentConfig defines Android emulator API URL
├─ WHO:   EnvironmentConfig.androidEmulatorApiUrl in lib/core/config/env
├─ CALLS: fromEnvironment('ANDROID_EMULATOR_API_URL', defaultValue: 'http:
└─ WHY:   Provides emulator-specific URL to reach host from Android emulat

STEP 3  DeviceCapabilityService detects Android emulator
├─ WHO:   DeviceCapabilityService.isSimulator in lib/core/services/devic
├─ CALLS: androidInfo() on DeviceInfoPlugin in lib/core/services/device_
└─ WHY:   Checks if running on Android emulator (isSimulator returns true

STEP 4  LlmPortFactory accesses emulator URL config
├─ WHO:   LlmPortFactory.create() in lib/core/llm/llm_port_factory.dart

```

```
└─ CALLS: EnvironmentConfig.androidEmulatorApiUrl() in lib/core/llm/llm_
└─ WHY:   Retrieves the emulator-specific URL for potential use in hybrid

STEP 5  BUG: Shared Dio in main.dart always uses apiUrl
└─ WHO:   setupDependencies() in lib/main.dart line 82
└─ CALLS: Dio(BaseOptions(baseUrl: EnvironmentConfig.apiUrl)) in lib
└─ WHY:   Central Dio instance uses default URL, bypassing emulator detect
! NOT WIRED: Android emulator URL selection not applied to shared Dio in
```

Key Files to Open

- lib/core/config/environment.dart
- lib/core/services/device_capability_service.dart
- lib/core/llm/llm_port_factory.dart
- lib/main.dart

Status Badge

 **PARTIALLY WIRED**

Gaps

 **GAP:** Android emulator URL resolution bypassed by shared Dio instance — Fix: In `setupDependencies()` in `main.dart`, conditionally set `baseUrl` to `EnvironmentConfig.androidEmulatorApiUrl` when `DeviceCapabilityService.isSimulator()` is true, or inject the correct URL into `CloudLlmAdapter` at construction time.

Feature 9 — Note Sync to Backend

Plain English Summary

When a note is saved or updated, the app should sync it to the backend API. The `NoteEditorCubit` triggers sync via `NoteSyncRepository`, which calls `NoteRemoteDatasource.syncNote()`. However, the actual HTTP POST to `/api/doctor-notes/sync` is commented out, and the corresponding backend route does not exist.

Call Chain


```
STEP 1 NoteEditorCubit loads or creates note and triggers sync
├ WHO: NoteEditorCubit.loadOrCreateNote() in lib/features/note_assist/
├ CALLS: _triggerSync() on NoteEditorCubit in lib/features/note_assist/
└ WHY: Initial note creation should sync to backend if connectivity is

STEP 2 NoteEditorCubit calls sync repository
├ WHO: NoteEditorCubit._triggerSync() in lib/features/note_assist/pre
├ CALLS: syncNoteToBackend() on NoteSyncRepository in lib/features/note
└ WHY: Delegates sync logic to repository layer.

STEP 3 NoteSyncRepository calls remote data source
├ WHO: NoteSyncRepository.syncNoteToBackend() in lib/features/note_as
├ CALLS: syncNote() on NoteRemoteDatasource in lib/features/note_assist
└ WHY: Repository delegates to data source for actual HTTP calls.

STEP 4 NoteRemoteDatasource serializes note for sync
├ WHO: NoteRemoteDatasource.syncNote() in lib/features/note_assist/da
├ CALLS: toJson() on ExtractedFields in lib/features/note_assist/data/r
└ WHY: Prepares note data for HTTP POST body.

STEP 5 HTTP POST to backend sync endpoint is commented out
├ WHO: NoteRemoteDatasource.syncNote() in lib/features/note_assist/da
⚠ NOT WIRED: Dio.post('/api/doctor-notes/sync', data: body) is commented
⚠ NOT WIRED: Backend route routes/api/doctor-notes/sync.dart does not exi
```

Key Files to Open

- lib/features/note_assist/presentation/cubit/note_editor_cubit.dart
- lib/features/note_assist/data/remote/note_sync_repository.dart
- lib/features/note_assist/data/remote/note_remote_datasource.dart

Status Badge

 NOT WIRED

Gaps

 GAP: Dio.post() for note sync is commented out in note_remote_datasource.dart line 32 — Fix: Uncomment the Dio.post() call and ensure the Dio instance is available (pass via constructor if needed).

 GAP: Backend route for note sync is missing — Fix: Create routes/api/doctor-

`notes/sync.dart` in the Dart Frog backend with a handler that receives note data and persists it to a database (not currently implemented).

Feature 10 — Retry & Circuit Breaker

Plain English Summary

The app uses a `RetryInterceptor` attached to the shared Dio instance to automatically retry transient network failures (connection timeouts, 5xx errors) with exponential backoff. A `CircuitBreaker` is registered in `GetIt` but is never attached to Dio or used to wrap any remote API calls, so it provides no runtime protection.

Call Chain

```
STEP 1  CircuitBreaker instantiated and registered in GetIt
├─ WHO:  setupDependencies() in lib/main.dart line 79
├─ CALLS: CircuitBreaker() constructor in lib/main.dart line 79
└─ WHY:  Creates circuit breaker instance for potential use in fault tolerance

STEP 2  RetryInterceptor attached to shared Dio
├─ WHO:  setupDependencies() in lib/main.dart line 90
├─ CALLS: dio.interceptors.add(RetryInterceptor(dio)) in lib/main.dart
└─ WHY:  Ensures transient network failures are automatically retried.

STEP 3  RetryInterceptor checks if error is retryable
├─ WHO:  RetryInterceptor.onError() in lib/core/network/retry_interceptor.dart
├─ CALLS: _shouldRetry(err) on RetryInterceptor in lib/core/network/retry_interceptor.dart
└─ WHY:  Determines if the failed request can be safely retried.

STEP 4  _shouldRetry checks HTTP status codes and methods
├─ WHO:  RetryInterceptor._shouldRetry() in lib/core/network/retry_interceptor.dart
├─ CALLS: statusCode check (5xx, connection timeout, receive timeout) in lib/core/network/retry_interceptor.dart
├─ CALLS: _retryableMethods.contains(method) in lib/core/network/retry_interceptor.dart
└─ WHY:  Retries GET, HEAD, OPTIONS, PUT, DELETE by default; POST only if idempotent

STEP 5  RetryInterceptor replays request on retryable errors
├─ WHO:  RetryInterceptor.onError() in lib/core/network/retry_interceptor.dart
├─ CALLS: Dio.fetch() in lib/core/network/retry_interceptor.dart line 5
└─ WHY:  Re-executes the failed request after backoff delay.

STEP 6  CircuitBreaker logic exists but is unused
```

└─ WHO: CircuitBreaker.call() in lib/core/network/circuit_breaker.dart

└─ CALLS: throws CircuitBreakerOpenException() in lib/core/network/circu

└─ WHY: Would prevent requests when circuit is open (after repeated fail

⚠ NOT WIRED: CircuitBreaker is registered in GetIt (lib/main.dart line 79

STEP 7 CircuitBreaker cooldown tracking exists

└─ WHO: CircuitBreaker._onFailure() in lib/core/network/circuit_breake

└─ CALLS: DateTime.now() in lib/core/network/circuit_breaker.dart line

└─ WHY: Sets cooldown period after which circuit transitions from open t

Key Files to Open

- lib/core/network/retry_interceptor.dart
- lib/core/network/circuit_breaker.dart
- lib/main.dart

Status Badge

⚠ PARTIALLY WIRED

Gaps

🔧 GAP: CircuitBreaker is registered but never used — Fix: Attach CircuitBreaker to Dio as an interceptor in main.dart line 90 alongside RetryInterceptor, or wrap CloudLlmAdapter's Dio calls with CircuitBreaker.call().

Section 6: Known Gaps — Consolidated Fix Plan

ID	Feature	What is missing	File where fix
GAP-1	2 — Clinical Processing Dispatch	No ClinicalProcessingRemoteDatasource; dispatch via CloudLlmAdapter only	lib/feature
GAP-2	4 — LLM Adapter Selection	Misleading comment in middleware	routes/_mid

GAP-3	5 — Response Propagation	<code>ClinicalProcessingResponse.fromJson()</code> missing	<code>lib/core/dt</code>
GAP-4	5 — Response Propagation	CloudLlmAdapter bypasses DTO parsing	<code>lib/core/ll</code>
GAP-5	6 — On-Device LLM Path	Android model verification is TODO	<code>lib/feature</code> line 194
GAP-6	6 — On-Device LLM Path	Android model download is commented out	<code>lib/feature</code> lines 210-221
GAP-7	7 — Note Persistence	No dedicated Drift DAO class	<code>lib/feature</code>
GAP-8	8 — Android Emulator URL	Shared Dio always uses <code>apiBaseUrl</code>	<code>lib/main.da</code>
GAP-9	9 — Note Sync	<code>Dio.post()</code> for sync commented out	<code>lib/feature</code> 32
GAP-10	9 — Note Sync	Backend route missing	<code>routes/api/</code>
GAP-11	10 — Retry & Circuit Breaker	CircuitBreaker never attached to Dio	<code>lib/main.da</code>

Priority Legend:

- **P0:** Blocks end-to-end testing or core functionality. Must fix before any integration testing.

- **P1:** Needed for full feature functionality. Should fix before Milestone 2.
- **P2:** Technical debt / nice-to-have. Can defer past initial release.

Effort Legend:

- **S:** < 1 hour (simple uncomment, add method, or configuration change)
- **M:** 1-4 hours (new file, moderate logic, or integration)
- **L:** > 4 hours (significant new feature or refactor)

Section 7: Key Files Quick Reference

File	Purp
<code>lib/main.dart</code>	App setu base
<code>lib/core/config/environment.dart</code>	Envir conf clou andr
<code>lib/core/llm/hybrid_llm_adapter.dart</code>	Mani nativ imple
<code>lib/core/llm/cloud_llm_adapter.dart</code>	Clou back

	proc
<code>lib/core/llm/llm_port_factory.dart</code>	Facto (Hyb envir conf
<code>lib/core/llm/android_native_llm_adapter.dart</code>	Andr using on-d
<code>lib/core/network/retry_interceptor.dart</code>	Dio i retrie
<code>lib/core/network/circuit_breaker.dart</code>	Circu (regi
<code>lib/core/ports/llm_port.dart</code>	LLM proc Proc
<code>lib/features/note_assist/presentation/pages/note_editor_page.dart</code>	Main micro Sugg
<code>lib/features/note_assist/presentation/cubit/ai_assist_cubit.dart</code>	Mani state prog
	Mani

<code>lib/features/note_assist/presentation/cubit/note_editor_cubit.dart</code>	auto
<code>lib/features/note_assist/data/services/on_device_llm_service.dart</code>	Dom proc LlmF
<code>lib/features/note_assist/data/local/note_local_repository.dart</code>	Drift savr entit
<code>lib/features/note_assist/data/remote/note_remote_datasource.dart</code>	Rem sync
<code>lib/core/services/local_speech_service.dart</code>	spee wrap reco
<code>routes/_middleware.dart</code>	Dart prov Clinic and I
<code>routes/api/v1/clinical-processing/process.dart</code>	Dart clinic
<code>lib/application/services/clinical_processing_orchestrator.dart</code>	Orch proc by Pi
<code>lib/infrastructure/llm/ollama_llm_adapter.dart</code>	Ollar clien

Section 8: Running the Project Locally

Prerequisites Checklist

- ☐ **Dart SDK 3.x** installed
- ☐ **Flutter SDK 3.x** installed (for `doctor_app`)
- ☐ **Dart Frog CLI** installed (`dart pub global activate dart_frog_cli`)
- ☐ **Ollama** installed and running (optional, for local LLM; stub mode works without it)
- ☐ **Android Studio / Xcode** (for mobile app)
- ☐ **Android emulator** or physical device (for `doctor_app`)

Step 1: Start the Backend (clinical-intelligence-dart)

```
# Navigate to backend directory
cd /path/to/EdgeLLMHub/projects/apps/clinical-intelligence-dart

# Install dependencies
dart pub get

# Start Dart Frog dev server (defaults to port 8080)
dart_frog dev
```

Verify Backend is Running:

- Open `http://localhost:8080/api/v1/clinical-processing/process` in Postman/curl
- Send POST with body: `{"inputText": "Patient presents with fever", "processingMode": "cleanTranscript"}`
- Expect: 200 OK with `processedText` field (if Ollama is running) or error (if Ollama is not running)

Ollama Setup (Optional):


```
# Install Ollama (macOS/Linux)
curl -fsSL https://ollama.com/install.sh | sh

# Pull a compatible model (Gemma 2B recommended for testing)
ollama pull gemma:2b

# Start Ollama server
ollama serve

# Verify Ollama is running
curl http://localhost:11434/api/tags
# Should return: {"models":[{"name":"gemma:2b",...}]}
```

Stub Mode:

- If Ollama is not running, the backend will fail to process requests (no stub adapter is active in middleware)
- To enable stub: Modify `routes/_middleware.dart` line 26 to use `StubLlmAdapter()` instead of `OllamaLlmAdapter()`

Step 2: Start the Mobile App (doctor_app)

```
# Navigate to app directory
cd /path/to/EdgeLLMHub/projects/apps/doctor_app

# Install dependencies
flutter pub get

# Run on Android emulator (replace <emulator-name> with your AVD)
flutter run -d <emulator-name> \
  --dart-define=API_URL=http://10.0.2.2:8080 \
  --dart-define=CLOUD_LLM_ENABLED=true

# Run on physical Android device (use actual device IP)
flutter run -d <device-id> \
  --dart-define=API_URL=http://<your-local-ip>:8080 \
  --dart-define=CLOUD_LLM_ENABLED=true

# Run on iOS simulator
```

```
flutter run -d iPhone \  
  --dart-define=API_URL=http://localhost:8080 \  
  --dart-define=CLOUD_LLM_ENABLED=true
```

Android Emulator URL Note:

- Use `10.0.2.2` to reach `localhost` from Android emulator
- **Current Issue:** The app's shared Dio instance in `main.dart` does not automatically use the emulator URL (see GAP-8). Workaround: Manually set `--dart-define=API_URL=http://10.0.2.2:8080`

On-Device LLM (Android):

- Requires `flutter_gemma` plugin and Google AI Edge
- Models must be downloaded via the model manager (currently TODO/placeholder — see GAP-5, GAP-6)
- Set `--dart-define=CLOUD_LLM_ENABLED=false` to force on-device only

Stub Mode (Offline):

- Set `--dart-define=CLOUD_LLM_ENABLED=false` and do not start backend
- The app will use `StubLlmAdapter` for static responses

Step 3: Test the End-to-End Flow

1. **Start backend:** `dart_frog dev` in `clinical-intelligence-dart/`
2. **Start Ollama:** `ollama serve` (optional, or use stub)
3. **Run app:** `flutter run` with appropriate `--dart-define` flags
4. **Test voice capture:**
 - Tap microphone FAB in note editor
 - Speak a test phrase (e.g., "Patient has a cough")
 - Verify text appears in editor
5. **Test clinical processing:**
 - Tap "Clean up" in AI toolbar
 - Verify processed text appears in SuggestionPanel

- Tap "Accept" to merge into note

6. **Test auto-save:**

- Edit note text
- Close and reopen app
- Verify note is restored from Drift

Step 4: Verify Ollama vs Stub Mode

Mode	Backend Config	App Config	Expected Behavior
Ollama	OllamaLlmAdapter() in middleware	CLOUD_LLM_ENABLED=true	Backend sends request to local Ollama; app returns dynamic response
Stub	StubLlmAdapter() in middleware	Any	Backend returns static responses; no Ollama required
On-Device	Any	CLOUD_LLM_ENABLED=false	App uses AndroidNativeLlmAdapter (flutter_gemma) for inference; backend not called

Section 9: Glossary

Term	Definition	Where It's Used
BLoC	Business Logic Component — Flutter state management pattern using events, states, and a BLoC	doctor_app project

	class. Cubit is a simplified version with single state stream.	
Cubit	Simplified BLoC with a single state stream and no events. Used in EdgeLLMHub for state management (e.g., <code>NoteEditorCubit</code> , <code>AiAssistCubit</code>).	<code>doctor_app</code> <code>pr</code>
Clean Architecture	Software design philosophy with concentric layers (Presentation → Domain → Data) where inner layers have no knowledge of outer layers. Dependency rule: Inner → Outer only.	<code>doctor_app</code> <code>ar</code>
Hexagonal Architecture	AKA Ports-and-Adapters. Application core (domain + application) is isolated from external systems (infrastructure) via interfaces (ports) and implementations (adapters).	<code>clinical-inte</code>
LlmPort	Interface defining the LLM contract: <code>processText(inputText, ProcessingMode)</code> . Concrete adapters implement this.	<code>lib/core/port</code>
HybridLlmAdapter	LLM adapter that implements a fallback chain: tries native on-device first, then cloud (if enabled), then stub.	<code>lib/core/llm/</code>
ProcessingMode	Enum defining the type of clinical text processing: <code>cleanTranscript</code> (clean up raw	<code>lib/core/port</code> <code>clinical_proc</code>

	dictation), <code>vocabAssist</code> (medical terminology assistance).	
PHI	Protected Health Information — any data that can identify a patient. EdgeLLMHub prioritizes on-device processing to keep PHI local.	Core design pri
Drift	Reactive persistence library for Flutter/Dart. Generates type-safe SQLite database access code.	<code>doctor_app</code> d
Dart Frog	Lightweight Dart backend framework for building REST APIs. Uses <code>provider()</code> for dependency injection.	<code>clinical-inte</code>
<code>provider()</code>	Dart Frog's dependency injection mechanism. Services are provided in middleware and read from handlers via <code>context.read<T>()</code> .	<code>clinical-inte</code> and handlers
GetIt	Service locator / DI container for Dart/Flutter. Used in <code>doctor_app</code> to register and resolve dependencies.	<code>doctor_app</code> m
Dio	Powerful HTTP client for Dart. Used in <code>doctor_app</code> for backend API calls with interceptors.	<code>doctor_app</code> ne
CircuitBreaker	Fault tolerance pattern that "opens" after repeated failures, preventing further calls until a cooldown period passes.	<code>lib/core/netw</code>
	Dio interceptor that automatically	

RetryInterceptor	retries failed requests (based on HTTP method and status code) with exponential backoff.	lib/core/netw
DoctorNote	Domain entity representing a clinical note. Contains <code>rawText</code> , <code>consultationId</code> , <code>createdAt</code> , etc.	doctor_app dc
ClinicalProcessingRequest	DTO sent from app to backend. Contains <code>inputText</code> and <code>processingMode</code> .	clinical-inte
ClinicalProcessingResponse	DTO returned from backend to app. Contains <code>processedText</code> , <code>processingMode</code> , <code>warnings</code> , <code>generatedAt</code> , <code>metadata</code> .	clinical-inte
VOCAB_ASSIST	ProcessingMode for medical terminology assistance. Routes to <code>TerminologyAssistanceService</code> in backend.	Backend orches
CLEAN_TRANSCRIPT	ProcessingMode for cleaning up raw dictation into structured clinical notes. Routes to <code>TranscriptCleanupService</code> in backend.	Backend orches
On-device LLM	LLM inference happening locally on the mobile device (Android: Gemma via Google AI Edge; iOS: MLC). Preserves PHI by not sending data to cloud.	doctor_app LL
Cloud LLM fallback	When on-device LLM is unavailable and <code>cloudLlmEnabled</code> is true, the	HybridLlmAdap

	app falls back to calling the backend, which uses Ollama.	
Stub adapter	Fallback LLM adapter that returns static/predefined responses when both on-device and cloud are unavailable.	StubLlmAdapte

Something went wrong

PDF export failed: Attempting to parse an unsupported color function "color"

39 Min Read 7,759 Words 69,110 Chars All changes saved